

Documentación Completa del Proyecto: Asistente Financiero "Luca"

1. Resumen Ejecutivo

Luca es un asistente financiero personal conversacional diseñado para operar a través de WhatsApp (utilizando integraciones con Twilio o directamente con Meta Cloud API). Su objetivo principal es ayudar a los usuarios ecuatorianos a llevar un control de sus finanzas de manera amigable e inteligente.

El proyecto está diseñado bajo tres pilares (historias de usuario):

- **H1 (Gastos e Ingresos):** Registra y categoriza gastos e ingresos (puntuales o recurrentes como el sueldo).
- **H2 (Presupuestos e Insights):** Proporciona información sobre el presupuesto y el balance, con alertas proactivas.
- **H3 (Soporte RAG):** Responde preguntas de soporte basándose en una Base de Conocimiento (Knowledge Base o KB). Si detecta temas sensibles (reclamos, consejos de inversión, temas regulatorios), escala automáticamente el caso a un agente humano, garantizando la seguridad (Guardrail).

2. Stack Tecnológico y Frameworks

El sistema fue construido buscando simplicidad, bajo costo operativo y alta capacidad de respuesta.

- **Backend Framework: FastAPI** (Python ≥ 3.11). Se eligió por su rapidez, integración nativa con validaciones de datos (Pydantic) y generación automática de documentación.
- **Base de Datos: Supabase (PostgreSQL)**. Funciona como la única fuente de estado ("single source of truth"), haciendo que la aplicación en sí misma sea *stateless* (sin estado local), lo cual permite escalarla horizontalmente con facilidad.
- **Modelos de Inteligencia Artificial (LLMs):**
 - **Claude (Anthropic - claude-sonnet-5):** Es el "cerebro" principal. Maneja las conversaciones, extrae datos financieros, consulta el presupuesto y responde preguntas de soporte.
 - **Groq (gpt-oss-20b):** Se usa específicamente como un **Guardrail (Filtro de Seguridad)** ultrarrápido y económico para clasificar si un mensaje es sensible antes de que llegue a Claude.
- **Despliegue (Hosting):** La aplicación está empaquetada en contenedores (Docker) y desplegada en **Railway** (con historial de evaluación en Fly.io).
- **Gestor de dependencias: uv**, un gestor de paquetes de Python ultrarrápido (usa `uv.lock` y `pyproject.toml`).
- **Tareas Programadas (Scheduler): APScheduler**, para enviar recordatorios recurrentes de sueldos y alertas de presupuesto proactivas.

3. Arquitectura del Sistema

El proyecto implementa una **Arquitectura Hexagonal (Puertos y Adaptadores)**. ¿Qué significa esto en términos simples? Significa que el "núcleo" de la aplicación (las reglas de negocio) está completamente aislado del mundo exterior.

Si mañana queremos cambiar WhatsApp por Telegram, o cambiar de modelo de Inteligencia Artificial, solo necesitamos escribir un nuevo "adaptador" sin modificar ni una sola línea del código principal.

Estructura de Puertos (Interfaces):

1. **ChannelAdapter (Canales):** Recibe mensajes de WhatsApp (Meta o Twilio) o de un Chat Web, y los convierte a un formato estándar interno.
 2. **LLMProvider (IA):** Conecta el sistema con Claude o Groq.
 3. **AgentHandler (Agentes):** Enruta la intención del usuario a la lógica correcta (Gasto, Presupuesto, Soporte).
 4. **Repository (Persistencia):** Guarda y lee información de Supabase.
 5. **Guardrail (Seguridad):** Evalúa la sensibilidad del mensaje.
-

4. Flujo de Procesamiento: ¿Qué pasa cuando envías un mensaje?

Dado que WhatsApp exige que el sistema responda que recibió el mensaje (Status 200 OK) muy rápido, el sistema divide el trabajo en dos etapas:

Etapas 1: Pre-procesamiento (Síncrono y ultra rápido)

1. **Consentimiento Legal:** Si es un usuario nuevo, se le pide aceptar términos legales.
2. **Guardrail (Filtro de seguridad en capas):**
 - **Capa 1 (Lista negra):** Verifica si el mensaje tiene palabras prohibidas (ej. "demanda", "inversión").
 - **Capa 2 (Groq LLM):** El modelo ultrarrápido analiza la intención.
 - Si se detecta un tema sensible, se corta el flujo, se crea un ticket de soporte para un humano y se le avisa al usuario amablemente. El mensaje no llega a la IA principal.

Etapas 2: Trabajo del Agente (Asíncrono, en segundo plano)

1. **Memoria Semántica e Historial:** El sistema busca en la base de datos los últimos mensajes de la conversación y busca "recuerdos" o "hechos" guardados previamente (ej. "Mi sueldo cae los 30").
 2. **Inferencia (Claude):** Claude lee el contexto, usa "herramientas" (tools) como `registrar_gasto`, `consultar_presupuesto`, etc., interactúa con la base de datos, formula una respuesta y la envía por WhatsApp de regreso al usuario.
 3. **Auditoría:** Todo se guarda en la tabla de `messages` para dejar un rastro legal (Audit Trail) de qué decidió la IA.
-

5. Base de Datos y Tablas (Supabase)

El diseño es relacional y robusto. Las tablas principales son:

- `users` : Registra a los usuarios (teléfono, nombre, y si aceptaron los términos legales).
 - `messages` : Guarda el historial de chat y sirve como registro de auditoría (quién dijo qué, qué herramienta usó la IA y cuándo).
 - `categories` : Catálogo de categorías válidas para gastos e ingresos.
 - `transactions` : Centraliza tanto los gastos como los ingresos. Tiene una columna `tipo` (gasto o ingreso) y un `status` (por si falta que el usuario confirme un dato antes de guardarlo definitivamente).
 - `recurring_incomes` : Guarda la configuración de ingresos fijos (ej. un sueldo que entra el día 30 de cada mes).
 - `income_reminders` / `alerts` : Tablas de control para asegurar que el sistema no le envíe la misma alerta o recordatorio de sueldo dos veces el mismo mes.
 - `budgets` : Configuración de presupuestos del usuario.
 - `tickets` : Cuando el Guardrail detecta algo sensible, crea un registro aquí para que un agente humano lo atienda desde el Panel Web.
 - `auth_codes` y `sessions` : Tablas de seguridad para la autenticación en el panel web.
-

6. Evolución de la IA: Memoria Semántica y Agencia

El sistema incorpora un plan avanzado para hacer al bot más "inteligente" recordando cosas a largo plazo sin alucinar:

- **Búsqueda Vectorial (pgvector):** Se convierte el texto en vectores matemáticos para buscar mensajes antiguos por "similitud de significado" y no solo por palabras exactas.
 - **Hechos del Usuario (User Facts):** Un trabajo programado revisa las conversaciones y guarda datos clave ("Le gusta la pizza", "Su sueldo es de \$500") para inyectarlos en futuras conversaciones de forma dinámica.
 - **Resúmenes Episódicos:** Las sesiones viejas se resumen en pocas líneas para ahorrar tokens (costos) y memoria.
-

7. Panel de Control Humano y WebApp

Además de WhatsApp, el proyecto cuenta con:

1. **Panel Humano:** Una interfaz web sencilla (usando FastAPI, Jinja2 y HTMX/Tailwind) para que los operadores humanos vean la cola de tickets generada por el Guardrail y respondan directamente al usuario.
 2. **Chat Web (Plan B):** Una interfaz alternativa para usar a Luca desde el navegador web sin depender de WhatsApp, ideal para demostraciones.
-

8. Conclusión

Luca no es solo un script de OpenAI conectado a WhatsApp. Es un sistema de software robusto, con arquitectura de grado empresarial, defensas regulatorias integradas desde el diseño (Guardrails duros, sin depender solo del prompt) y preparado para escalar sin perder confiabilidad.